

UNIVERSIDAD TECNOLÓGICA DE PEREIRA
 Facultad de Ingenierías – Programa de Ingeniería de Sistemas y Computación
 IS873 – Laboratorio de Software (Semestre 2026-2)

NOTA 1 — DEFINICIÓN DEL PROYECTO Y PRUEBA DE CONCEPTO (PoC)

Sistema de Gestión y Compra de Libros en Línea

Equipo: Jose David Marin Giraldo • Omar David Bernal Restrepo • Andres Felipe Mora • Maximiliano Giraldo

<i>Asignatura / Ponderación</i>	<i>IS873 – Laboratorio de Software (30% de la nota final)</i>
<i>Modalidad / Fecha Sustentación</i>	<i>Sprint 0 / Demo Day de Validación (13 y 15 de octubre de 2026)</i>
<i>Framework Metodológico</i>	<i>Casos de Uso a 3 niveles de Cockburn + Priorización MoSCoW</i>
<i>Stack Tecnológico Propuesto</i>	<i>Python (FastAPI, APScheduler), PostgreSQL, React (TypeScript, Tailwind CSS), Google Maps Platform API</i>

1. Planteamiento del Proyecto

1.1 Problema Real

La comercialización de libros en librerías que integran venta física y canal digital enfrenta graves fallas operativas cuando no existe trazabilidad granular. La falta de control individualizado por copia física impide conocer la disponibilidad real en bodega o sucursales, generando sobreventas o inventarios fantasma. Asimismo, la ausencia de expiración en las reservas retiene stock crítico por tiempo indefinido, los canales carecen de recogida georreferenciada en tienda y las devoluciones se tramitan manualmente sin validación digital ni control temporal.

1.2 Usuarios Finales

- **Visitante:** Explora el catálogo público, realiza búsquedas y accede al registro.
- **Cliente:** Usuario autenticado con permisos para comprar, reservar libros (máx. 24h), tramitar devoluciones con QR, consultar envíos y chatear con administradores.
- **Administrador:** Gestiona libros y ejemplares físicos únicos, controla stock y atiende soporte. Tiene restricción estricta de compra/reserva.
- **Root (Superadmin):** Único facultado para crear/revocar cuentas de administrador y auditar la plataforma.
- **Sistema (Scheduler):** Worker asíncrono que libera reservas vencidas (24h) y emite bonos diarios.

1.3 Alcance (Incluido vs. Fuera de Alcance)

El MVP incluye: Autenticación RBAC segura (bcrypt/JWT), catálogo e inventario diferenciando Libro de Ejemplar único, búsqueda avanzada, carrito de compras con pasarela simulada (PCI-DSS), reservas temporales de 24h con liberación automática, logística con Google Maps y devoluciones con código QR (< 8 días). Queda explícitamente fuera de alcance: pasarela de pago bancaria real en producción, rastreo telemático GPS en tiempo real de vehículos, facturación electrónica DIAN y Realidad Aumentada.

1.4 Restricciones y Supuestos

Restricciones: 4 sprints académicos de desarrollo, uso exclusivo de infraestructura en capa gratuita (Google Maps Demo Key, Neon PostgreSQL, Vercel, Render) y cumplimiento de la Ley 1581 de 2012 (Habeas Data). Supuestos: Usuarios con navegadores web modernos y conexión a internet estable.

2. Levantamiento Completo de Requerimientos

2.1 Requerimientos Funcionales (39 RFs) — Formato 'El sistema debe...'

- **RF-001:** El sistema debe permitir que un usuario Administrador registre nuevos libros en el inventario con campos obligatorios completos.
- **RF-002:** El sistema debe asignar automáticamente un código único e irrepetible a cada ejemplar físico registrado.
- **RF-003:** El sistema debe permitir al Administrador modificar la información de libros registrados previamente.
- **RF-004:** El sistema debe permitir al Administrador dar de baja o eliminar ejemplares del inventario.
- **RF-005:** El sistema debe permitir al Administrador controlar y actualizar las existencias de stock físico en tiempo real.
- **RF-006:** El sistema debe mover automáticamente los libros con existencia cero a la categoría de Histórico Agotado.
- **RF-007:** El sistema debe agregar automáticamente cada libro registrado a la sección pública de noticias/novedades.
- **RF-008:** El sistema debe permitir a los usuarios registrados buscar libros disponibles para compra o reserva.

- **RF-009:** El sistema debe permitir a los clientes reservar ejemplares disponibles por un período máximo de 24 horas.
- **RF-010:** El sistema debe liberar automáticamente las reservas y reintegrar stock al cumplirse 24 horas sin confirmación de compra.
- **RF-011:** El sistema debe limitar las reservas activas a un máximo de 5 libros diferentes y 3 ejemplares por título por cliente.
- **RF-012:** El sistema debe permitir al cliente gestionar un carrito de compras interactivo antes de realizar el pago.
- **RF-013:** El sistema debe conectar el carrito con la pasarela financiera para procesar pagos con tarjeta o saldo disponible.
- **RF-014:** El sistema debe permitir al cliente cancelar una compra o reserva antes del despacho, revirtiendo el stock.
- **RF-015:** El sistema debe permitir al cliente consultar su historial detallado de compras realizadas y canceladas.
- **RF-016:** El sistema debe permitir al cliente solicitar devoluciones sobre libros comprados y entregados.
- **RF-017:** El sistema debe exigir la selección de una causa tipificada de devolución (mal estado, retraso, expectativa).
- **RF-018:** El sistema debe proveer un campo adicional para detallar el motivo de la devolución.
- **RF-019:** El sistema debe bloquear automáticamente solicitudes de devolución pasados 8 días calendario tras la entrega.
- **RF-020:** El sistema debe generar un código QR único y enviarlo por correo para la validación de la devolución.
- **RF-021:** El sistema debe registrar y permitir editar la dirección de correspondencia del cliente para envíos.
- **RF-022:** El sistema debe permitir al cliente consultar el estado del envío (EN PREPARACIÓN, ENVIADO, ENTREGADO).
- **RF-023:** El sistema debe ofrecer la opción de recogida en tienda física presencial para clientes en Colombia.
- **RF-024:** El sistema debe calcular automáticamente la tienda física más cercana con inventario disponible.
- **RF-025:** El sistema debe permitir visualizar sucursales en un mapa interactivo con Google Maps Platform API.
- **RF-026:** El sistema debe gestionar los roles Root, Administrador, Cliente y Visitante con permisos diferenciados (RBAC).
- **RF-027:** El sistema debe permitir el registro de clientes solicitando DNI, datos personales, correo único y contraseña cifrada.
- **RF-028:** El sistema debe permitir a los usuarios autenticados editar su información de perfil y contraseña.
- **RF-029:** El sistema debe permitir al cliente suscribirse al canal de noticias y seleccionar categorías favoritas.
- **RF-030:** El sistema debe permitir únicamente al usuario Root crear cuentas con rol Administrador.
- **RF-031:** El sistema debe impedir que usuarios con rol Root o Administrador realicen compras o reservas.
- **RF-032:** El sistema debe enviar automáticamente un bono de descuento válido por 1 día en el cumpleaños del cliente.
- **RF-033:** El sistema debe mostrar un catálogo de novedades literarias exclusivo para clientes suscritos.
- **RF-034:** El sistema debe proveer mensajería asíncrona en hilos para la interacción entre cliente y administrador.
- **RF-035:** El sistema debe permitir búsqueda avanzada por título, autor, género, editorial, idioma, ISSN y precio.
- **RF-036:** El sistema debe permitir registrar tarjetas de crédito/débito tokenizadas bajo estándares PCI-DSS.
- **RF-037:** El sistema debe actualizar automáticamente el saldo disponible del cliente tras compras o cancelaciones.
- **RF-038:** El sistema debe implementar un bot que recomiende lecturas según historial de compras y búsquedas.
- **RF-039:** El sistema debe proveer una interfaz para previsualización en Realidad Aumentada (opcional / deuda técnica).

2.2 Requerimientos No Funcionales (RNF)

- **RNF-01 (Rendimiento - Concurrencia):** Soportar al menos 300 usuarios concurrentes sin degradación del servicio.
- **RNF-02 (Rendimiento - Latencia):** El 95% de las transacciones debe responder en menos de 4.0 segundos bajo carga normal.
- **RNF-03 (Rendimiento - Asincronía):** La liberación de reservas (24h) y despacho de correos se procesarán en segundo plano.
- **RNF-04 (Seguridad - Credenciales):** Contraseñas cifradas con hashing bcrypt unidireccional.
- **RNF-05 (Seguridad - Comunicaciones):** Todo el tráfico web viaja obligatoriamente sobre HTTPS con TLS 1.3.
- **RNF-06 (Seguridad - PCI-DSS):** No almacenar datos bancarios sensibles; operar exclusivamente con tokens de pasarela.
- **RNF-07 (Seguridad - Inyecciones):** Protección estricta contra SQL Injection y XSS mediante ORM y validación Pydantic.
- **RNF-08 (Fiabilidad - Atomicidad):** Transacciones de compra y reserva completamente atómicas (ACID).
- **RNF-09 (Disponibilidad):** Garantizar uptime del 99.5% mensual (inactividad no planificada < 3.6 horas/mes).
- **RNF-10 (Usabilidad / Portabilidad):** Diseño Mobile-First responsivo compatible con Chrome, Firefox, Safari y Edge.
- **RNF-11 (Marco Legal):** Cumplimiento estricto de la Ley 1581 de 2012 de Colombia (Habeas Data) con consentimiento explícito.

2.3 Justificación Metodológica: Casos de Uso + MoSCoW

Se seleccionó el framework de Casos de Uso + MoSCoW porque modela con precisión los contratos de interacción y políticas de seguridad de 4 roles dispares, vincula de forma transparente las 34 entidades de base de datos normalizadas y proporciona una categorización binaria e innegociable ante el comité de inversión para salvaguardar el alcance del semestre.

3. Casos de Uso Críticos (Metodología Cockburn)

Se desarrollaron los 4 casos de uso más críticos del sistema a tres niveles: Resumen (proceso de negocio), Objetivo de Usuario (interacción en sesión) y Subfunción (paso atómico técnico).

3.1 Diagrama General de Casos de Uso (Relación Actores – Sistema)

A continuación se presenta el mapa general de casos de uso del sistema. Cada actor cuenta con una convención cromática diferenciada en sus líneas de asociación para garantizar absoluta claridad visual y evitar interpretaciones ambiguas:

- **Visitante (Púrpura):** Interacción con módulos públicos de registro y consulta básica.

- **Cliente (Azul):** Acceso a compras, reservas, carrito, devoluciones y gestión de perfil.
- **Administrador (Verde):** Control granular de inventario, gestión de libros y soporte técnico.
- **Root (Rojo):** Administración de cuentas del personal, privilegios y auditoría de seguridad.
- **Scheduler / Sistema (Naranja):** Procesos automatizados de liberación de reservas a las 24 horas y emisión de bonos.

IS873 – Laboratorio de Software

UNIVERSIDAD TECNOLÓGICA DE PEREIRA
Facultad de Ingenierías – Programa de Ingeniería de Sistemas y Computación
IS873 – Laboratorio de Software

Proyecto 2: Sistema de Gestión y Compra de Libros en Línea

Plataforma web integral para catálogo, venta, reserva de ejemplares, logística y recomendaciones

Tipo de asignatura	Práctica (Laboratorio de Software)
Stack sugerido	Python (FastAPI, SQLAlchemy, Pydantic, APScheduler), PostgreSQL, React (TypeScript, Tailwind CSS), Google Maps Platform API, pasarela simulada (PCI-DSS tokenización), generación de QR.

1. Planteamiento del problema

La gestión y comercialización de libros en librerías físicas y virtuales suele enfrentar serias fallas de sincronización y control de existencias. La ausencia de un seguimiento individual por copia física impide conocer la disponibilidad real en bodega o sucursales, ocasionando sobreventas o inventarios fantasma. Paralelamente, los usuarios apartan libros de forma indefinida sin concretar la adquisición, bloqueando stock crítico por falta de un mecanismo automatizado de expiración. Asimismo, los canales tradicionales carecen de logística geolocalizada para recogida en tienda, procedimientos de devolución ágiles con validación digital y personalización basada en hábitos lectores.

2. Objetivo general

Desarrollar una plataforma web integral para la administración, reserva, comercialización y distribución de libros, garantizando la trazabilidad granular de cada ejemplar mediante códigos únicos, compras seguras, control de reservas temporales con liberación automática, gestión de envíos georreferenciados, devoluciones mediante códigos QR y un asistente inteligente de recomendaciones.

3. Objetivos específicos

- Implementar autenticación robusta y control de acceso basado en roles (RBAC: Root, Administrador, Cliente y Visitante) con credenciales cifradas (bcrypt) y tokens JWT.
- Diseñar el catálogo e inventario físico diferenciando la información bibliográfica (Libro) de la copia individual (Ejemplar), con código único y gestión de agotados.
- Construir el flujo transaccional de compras (carrito persistente, checkout con tokenización PCI-DSS y saldo) y reservas con caducidad automática de 24 horas y liberación de stock.
- Integrar la logística de envíos con trazabilidad de estados y opción de recogida en tienda física geolocalizada mediante la API de Google Maps.
- Crear un módulo de postventa para cancelaciones con reintegro y devoluciones (< 8 días) validadas mediante código QR emitido por correo electrónico.
- Incorporar servicios de valor agregado: bonos automáticos de cumpleaños, boletín de novedades a suscriptores, mensajería cliente-administrador y bot de recomendaciones.

4. Alcance del MVP

- Gestión de usuarios y roles: Registro con correo único, inicio de sesión seguro con JWT, gestión de roles, edición de perfil y creación exclusiva de administradores por Root.
- Inventario y catálogo: Registro de libros y copias con código único por ejemplar, búsqueda avanzada multicriterio y publicación automática en novedades.
- Compras y reservas: Carrito persistente, checkout con pasarela simulada (PCI-DSS), orden de compra, y reservas temporales de hasta 24h con liberación automática de stock.
- Logística y entregas: Trazabilidad del envío (En preparación, Enviado, Entregado) y selección de tienda física en mapa interactivo de Google Maps.
- Cancelaciones y devoluciones: Cancelación antes del despacho con reintegro de saldo; solicitud de devolución (< 8 días) con código QR emitido por correo electrónico.
- Fidelización y comunicación: Tarea programada diaria para emisión de bono de cumpleaños, mensajería cliente-admin y bot de recomendación según historial.

Página 1

Figura 3.1: Diagrama General de Casos de Uso — Módulos de Clientes, Visitantes y Administración (Parte 1)

5. Fuera de alcance (explícito)

Cobro real de dinero bancario en producción (se operará con pasarela en entorno sandbox/simulado con cumplimiento conceptual de PCI-DSS); rastreo telemático GPS en tiempo real de vehículos de mensajería; facturación electrónica tributaria oficial (DIAN); visualización de modelos 3D con Realidad Aumentada (identificado como requerimiento opcional de baja prioridad en el Sprint 4).

6. Stack tecnológico sugerido

Python, FastAPI, SQLAlchemy, Pydantic, APScheduler para tareas periódicas, PostgreSQL para base de datos relacional (34 entidades), React (Vite, TypeScript, Tailwind CSS) para frontend responsivo, Google Maps Platform API para geolocalización de tiendas, librería qrcode/Pillow para generación de códigos QR y JWT para autenticación segura.

7. Herramientas e insumos sugeridos (gratuitos)

- **Google Maps Platform (Demo Key / Free Tier)** – Visualización de sucursales en mapa interactivo y cálculo de tienda más cercana sin costos de facturación.
- **Neon.tech o Supabase (capa gratuita)** – Base de datos PostgreSQL administrada en la nube con soporte nativo para transacciones concurrentes.
- **Render o Railway (planes free)** – Despliegue de la API en FastAPI y workers de tareas periódicas con APScheduler.
- **Vercel o Netlify (capa gratuita)** – Despliegue y hosting continuo del frontend en React con integración directa a GitHub.
- **Postman o Insomnia** – Pruebas y validación de endpoints de la API REST (autenticación, reservas, carrito, pedidos).
- **GitHub & GitHub Projects** – Control de versiones, ramas por funcionalidad, Pull Requests con Definition of Done y gestión del backlog Scrum.

8. Riesgos y mitigaciones

- **Condición de carrera y sobreventa de ejemplares simultáneos** – Mitigar con transacciones atómicas en PostgreSQL y bloqueos a nivel de fila (SELECT FOR UPDATE) durante el checkout y la reserva.
- **Fallo o reinicio del servidor durante reservas activas** – Mitigar persistiendo la fecha_vencimiento en base de datos y ejecutando un job periódico en APScheduler que verifique registros expirados en lote.
- **Riesgo de seguridad con datos bancarios** – Mitigar aplicando tokenización estricta mediante pasarela simulada, impidiendo el almacenamiento de números de tarjeta o CVV en la base de datos local (PCI-DSS).
- **Dependencia de límites de cuota en Google Maps API** – Mitigar precargando las coordenadas de sucursales en la base de datos y renderizando mapas interactivos con marcadores locales.

9. Entregables esperados

Documento formal de requerimientos (39 RFs) y matriz de casos de uso (CU-01 a CU-16); actas de los 4 Sprints con historias de usuario (HU-1 a HU-35) y Definition of Done cumplida; documento de diseño de base de datos y diagramas de clases UML (DC-01 a DC-09); repositorio de GitHub con código fuente, suite de pruebas unitarias/integración (cobertura >= 80%); API REST documentada en OpenAPI/Swagger; aplicación web funcional desplegada en la nube; manual de usuario y despliegue; demostración funcional final aprobada por el equipo docente.

Figura 3.2: Diagrama General de Casos de Uso — Módulos de Root, Tareas Programadas y Logística (Parte 2)

3.2 CU-01: Comprar Libro (Checkout Transaccional)

- Resumen: Compra integral de libros agregados al carrito con deducción en tiempo real y pago seguro tokenizado.
- Usuario: El cliente valida carrito, selecciona entrega (domicilio o tienda) y confirma pago con saldo o tarjeta. Flujo alterno: Si la pasarela rechaza la tarjeta o el saldo es insuficiente, se aplica rollback atómico.
- Subfunción SF-01: Bloqueo de fila (SELECT FOR UPDATE) sobre EJEMPLAR, cambio a estado 'VENDIDO' y confirmación de transacción.

3.3 CU-02: Reservar Libro y Liberación Automática a las 24 Horas

- Resumen: Bloqueo temporal de copias físicas por máximo 24h, liberando el stock si no se compra.
- Usuario: Cliente aparta libro (máx. 5 títulos / 3 ejemplares). Flujo de Excepción: El Scheduler (APScheduler) revisa cada 15 min reservas vencidas, expira el registro y regresa el ejemplar a 'DISPONIBLE'.
- Subfunción SF-02: Actualización masiva en lote de ejemplares expirados y recálculo de stock.

3.4 CU-03: Recogida en Tienda Física con Geolocalización (Google Maps)

- Resumen: Retiro en sucursal física con ubicación en mapa y cálculo de tienda más cercana con inventario.

- Usuario: Cliente selecciona 'Recogida', visualiza mapa interactivo de Google Maps y elige tienda con stock. Excepción: Si la tienda carece de existencias completas, el sistema sugiere la sucursal más cercana con stock.
- Subfunción SF-03: Cálculo geoespacial de distancia a tiendas con inventario positivo.

3.5 CU-04: Solicitud de Devolución con Código QR y Límite de 8 Días

- Resumen: Tramitación de garantía para pedidos entregados con salvoconducto digital en código QR.
- Usuario: Cliente selecciona pedido entregado, diligencia causas y recibe QR por email. Excepción: Si fecha_actual - fecha_entrega > 8 días, el sistema rechaza la solicitud automáticamente.
- Subfunción SF-04: Generación y firma criptográfica del token QR (HMAC-SHA256) codificado en imagen PNG.

4. Priorización MoSCoW y Defensa de los Must Have

La clasificación del backlog protege al equipo ante restricciones temporales:

- **MUST HAVE (17 RFs — 43.6%):** RF-001, 002, 003, 004, 005 (Inventario granular), RF-008, 009, 010, 011, 012, 013, 014 (Compras y reservas de 24h), RF-026, 027, 028, 030, 031 (Seguridad RBAC). Sin ellos no existe producto.
- **SHOULD HAVE (12 RFs — 30.8%):** RF-006, 015, 016, 017, 019, 020, 021, 022, 023, 024, 035, 036 (Historial, Devoluciones QR, Tiendas físicas).
- **COULD HAVE (7 RFs — 17.9%):** RF-007, 018, 025, 029, 032, 033, 034, 037, 038 (Mapa interactivo, Bonos de cumpleaños, Chat, Recomendaciones).
- **WON'T HAVE (3 RFs — 7.7%):** RF-039 (Realidad Aumentada), Pasarela bancaria con dinero real, Rastreo telemático GPS de camiones.

4.1 Defensa de los 'Must Have' ante el Comité de Inversión

1. Sin inventario con código único por copia física (RF-001/002/005), la librería no tiene producto que comercializar y sufriría inventarios fantasma.
2. Sin compras transaccionales atómicas (RF-008/012/013), el negocio no monetiza.
3. La liberación automática de reservas a las 24h (RF-009/010) es indispensable: sin ella, los clientes apartarían libros indefinidamente, bloqueando las ventas de la empresa.
4. La seguridad RBAC (RF-026/030/031) garantiza que los administradores no puedan comprarse libros a sí mismos ni manipular inventarios sin auditoría.

5. Diseño Preliminar de la Solución

5.1 Arquitectura C4 y Modelo de Datos

Se estructuró la solución en C4 Nivel 1 (Contexto de Sistema con usuarios y servicios externos: Google Maps Platform, Pasarela de Pagos Tokenizada y Servidor SMTP de Notificaciones) y C4 Nivel 2 (Contenedores: Frontend SPA React con Vite y TypeScript, Backend REST FastAPI en Python, Base de Datos relacional PostgreSQL con 34 tablas normalizadas y Worker en segundo plano APScheduler para liberación periódica de stock).

5.2 Modelo de Datos Relacional (34 Entidades)

El modelo relacional segrega estrictamente el concepto bibliográfico genérico (LIBRO) del objeto físico comercializable (EJEMPLAR), permitiendo que cada copia posea su propio código de barras, sucursal física y estado (DISPONIBLE, RESERVADO, VENDIDO, EN_DEVOLUCION, DADO_DE_BAJA). Las tablas transaccionales RESERVA y DETALLE_VENTA aseguran integridad referencial y atomicidad mediante claves foráneas y restricciones de unicidad.

5.3 Mockups y Wireframes de Pantallas / Flujos Críticos

A continuación se integran los mockups de diseño preliminar correspondientes a las pantallas y flujos más críticos del sistema, validados contra las Historias de Usuario del proyecto:

A. Búsqueda de Catálogo y Reserva de Ejemplares (CU-02 / RF-008, RF-009, RF-011, RF-035):

Permite consultar la ficha completa del libro, autores, categorías y verificar la disponibilidad de copias en tiempo real por sucursal. El botón 'Reservar' bloquea el ejemplar por máximo 24 horas:



Figura 5.1: Mockup — Detalle del Libro y Acción de Reserva de Ejemplar Físico

B. Carrito de Compras y Resumen de Orden (CU-01 / RF-012, RF-013, RF-036, RF-037):

Permite revisar la selección de libros, cantidades, subtotales, saldo disponible del monedero del cliente y proceder al checkout transaccional:

Carrito

	Item 1	Cant 1	Precio \$1000
	Item 2	Cant 1	Precio \$500
	Item 3	Cant 1	Precio \$25.32
Total \$1525.32			

[Volver a la tienda](#)

Figura 5.2: Mockup — Carrito de Compras, Desglose de Costos y Saldo Disponible

C. Solicitud de Devolución y Código QR Digital (CU-04 / RF-016 a RF-020):

Interfaz post-venta donde el cliente selecciona un pedido entregado, ingresa la causa tipificada y recibe el código QR que acredita su autorización antes del vencimiento del límite de 8 días:

Figura 5.3: Mockup — Módulo de Devoluciones y Salvoconducto Digital con Código QR

D. Panel Administrativo de Inventario y Stock Multisede (RF-001 a RF-005, RF-031):

Vista reservada para Administradores con tabla consolidada de inventario, filtros por categoría, actualización de unidades, botón para registrar ejemplares y restricción de compra/reserva para este perfil:

Figura 5.4: Mockup — Gestión de Inventario, Bajas y Existencias para Administradores

E. Recogida en Tienda Física y Geolocalización (CU-03 / RF-023, RF-024, RF-025):

Pantalla de selección de entrega que renderiza el mapa interactivo vía Google Maps Platform API mostrando sucursales de Pereira y Bogotá con inventario en stock:



Figura 5.5: Mockup — Mapa Interactivo de Google Maps para Recogida en Tienda

5.4 Justificación Técnica del Stack Elegido

- **Backend (Python / FastAPI):** Ofrece alto rendimiento asíncrono (ASGI), tipado estricto con Pydantic y documentación interactiva Swagger OpenAPI lista para pruebas sin configuración extra.
- **Base de Datos (PostgreSQL):** Cumplimiento estricto de ACID, soporte para bloqueos a nivel de fila (SELECT ... FOR UPDATE) indispensables para evitar sobreventas, y manejo eficiente de 34 tablas relacionales.
- **Frontend (React + Vite + Tailwind CSS):** Arquitectura basada en componentes reutilizables, reactividad fluida para el carrito y estilización responsiva rápida que facilita el trabajo simultáneo del equipo.
- **Scheduler (APScheduler en Python):** Garantiza la liberación periódica de reservas vencidas (24h) de manera liviana dentro del mismo ecosistema de la aplicación, evitando dependencias externas complejas como Redis o Celery.
- **API de Mapas (Google Maps Platform):** Permite geocodificación, renderizado de mapas interactivos y cálculo de distancia euclidiana/vial para retiro en sucursal física (aprovechando el Maps Demo Key para prototipado académico).

6. Product Backlog Inicial Priorizado (Sprints 1 y 2)

Estimado en Story Points mediante serie Fibonacci:

- **Sprint 1 (21 SP):** HU-01 Registro (5), HU-04 Login JWT (3), HU-05 Logout (1), HU-06 Cambio Contraseña (2), HU-07 Perfil (3), HU-02 Admin por Root (5), HU-03 Restricción Compra Admin (2).
- **Sprint 2 (27 SP):** HU-10 Registro Libro (5), HU-10b Código Único Ejemplar (5), HU-12 Edición Libro (3), HU-11 Baja Ejemplar (2), HU-13 Control Stock (3), HU-14 Vista Inventario Agotados (5), HU-09 Preferencias (2), HU-15 Novedades (2).

7. Prueba de Concepto (PoC) — Supuesto Técnico Más Riesgoso

Se identificó como riesgo crítico: '¿Es posible garantizar consistencia atómica y evitar sobreventas bajo alta concurrencia al reservar ejemplares únicos, mientras un scheduler asíncrono libera automáticamente el stock a las 24 horas?'

Se construyó el script funcional poc_reserva_concurrencia.py en la carpeta PoC_Riesgo_Tecnico/, donde 10 hilos concurrentes compitieron por un libro con 2 copias disponibles: 2 fueron aprobadas, 8 rechazadas limpiamente en < 30 ms, y el worker liberó el stock automáticamente al expirar.

8. Guía de Respuestas para la Sustentación (Demo Day)

- **1. Problema y Usuario:** Desincronización de inventario por copia física y retención de stock por reservas sin caducidad. Usuario: lectores/compradores y administradores de librería.
- **2. Historia de mayor riesgo:** HU-18 / RF-009/010 (Reserva concurrente y liberación a las 24h) por el riesgo de sobreventas e inconsistencia de stock.
- **3. Prueba de concepto:** Demuestra transacciones atómicas y scheduler autónomo. NO demuestra frontend React ni pasarela real.
- **4. Recorte de alcance si el tiempo apremia:** Se recortarían los ítems Could Have (Realidad Aumentada, Chat, Recomendaciones y Bonos de cumpleaños), blindando los 17 Must Have.
- **5. Framework de requerimientos:** Casos de Uso + MoSCoW por la granularidad de roles RBAC y su trazabilidad con las 34 entidades relacionales.
- **6. Próximo incremento demostrable (Nota 2):** Módulo de autenticación RBAC funcional y CRUD completo de inventario con control individual por copia física.

